

Supervised Learning for Machine Condition

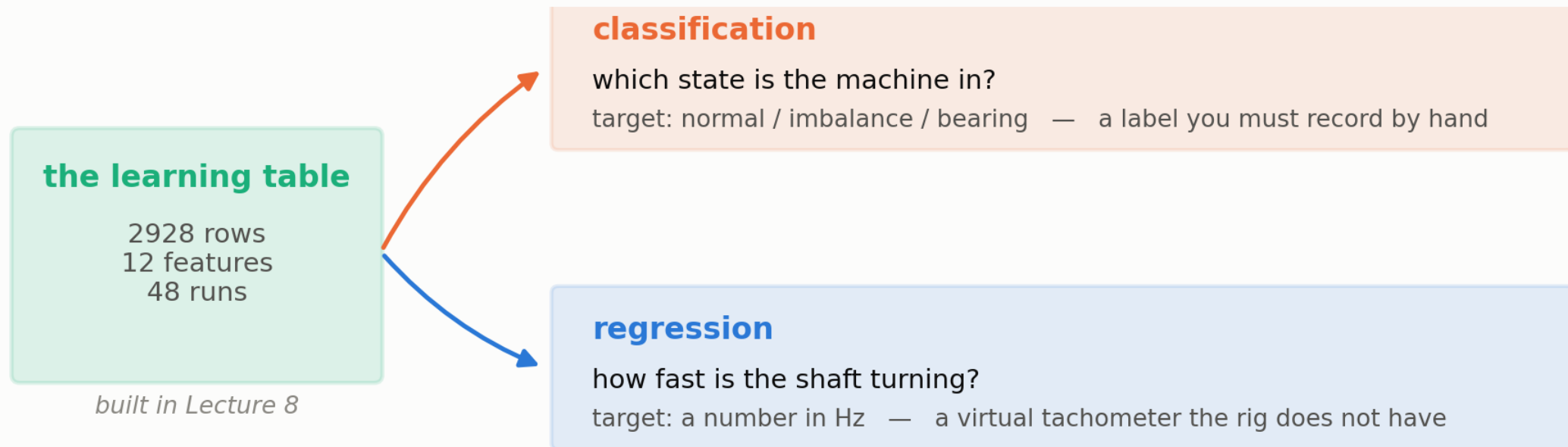
Five models on one dataset — and only two of them can ever leave the laptop

- Regression and classification — the same table, a different target column
- Five model families, and the shape of the boundary each one can draw
- Underfitting, overfitting, learning curves and regularisation
- Train, validate, test — a protocol that does not lie to you
- Accuracy is half the answer; size and inference cost are the other half

Notebook: `iaiiot26_Lect09_nb.ipynb`

Two questions, one table

- Same windows, same twelve features, same rule about splitting by run — only the target column changes.

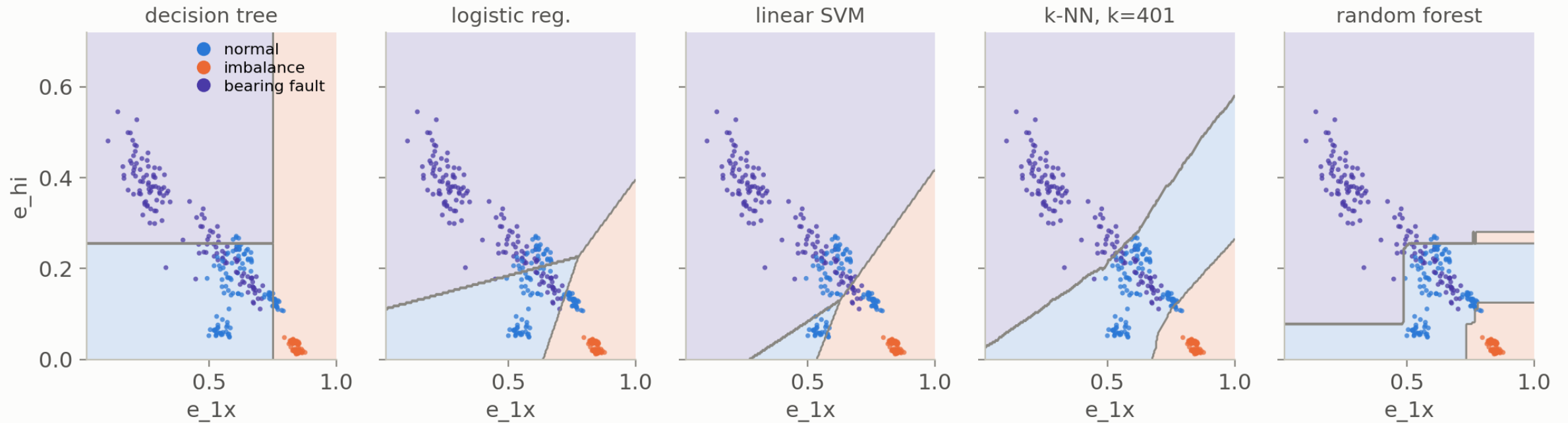


Same features, same windows, same split rules — only the target column changes.

Before you train anything, write down what a useless model scores: 0.333 for three balanced classes, and the standard deviation of the target for regression.

Five candidates, five shapes

- Each family can only draw certain boundaries. That restriction is not a defect — it is the prior knowledge the model brings.



*Trained on two features so the boundary can be drawn.
Axis-aligned steps, one straight cut, or a wandering contour — the shape is the model's inductive bias.*

Four of these five compress the training data into a handful of numbers. k-NN does not compress at all — it is the training data, and slide 9 sends the bill.

Three piles, not two

- Try nine tree depths on the test set and keep the best, and your reported number is the maximum of nine noisy measurements — biased upward by construction.

48 acquisition runs



1. fit

Train each candidate on the training runs only.

2. choose

Score every hyperparameter on the validation runs. Keep the winner.

3. report

Score that one model on the test runs. Once. Then stop.

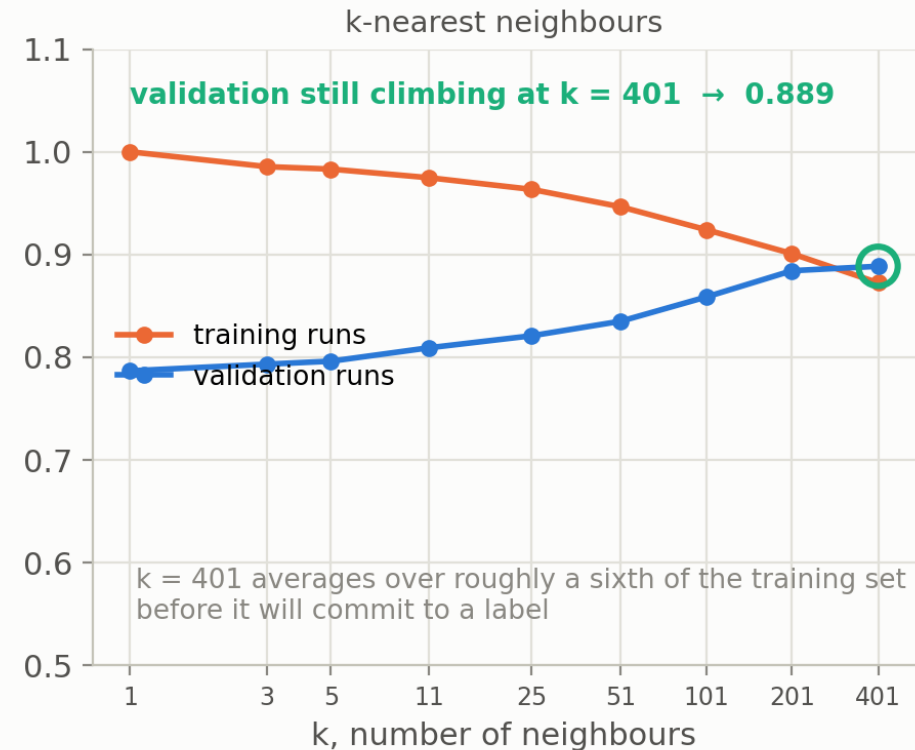
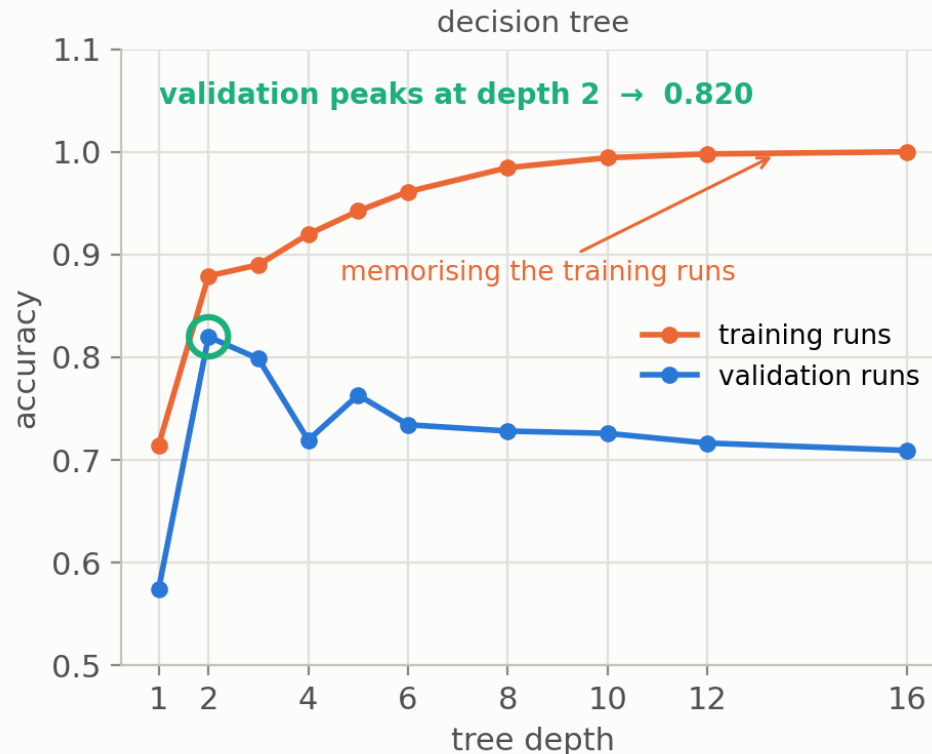


Score the test set more than once and it quietly becomes a second validation set.

48 runs, split 28 / 10 / 10. Fit on train, choose on validation, report on test — once. A test set you have optimised against is a validation set.

Too simple, too complex, and the middle

- Underfitting: both curves low and together. Overfitting: a wide gap that does not close. Only the validation curve tells you which one you have.

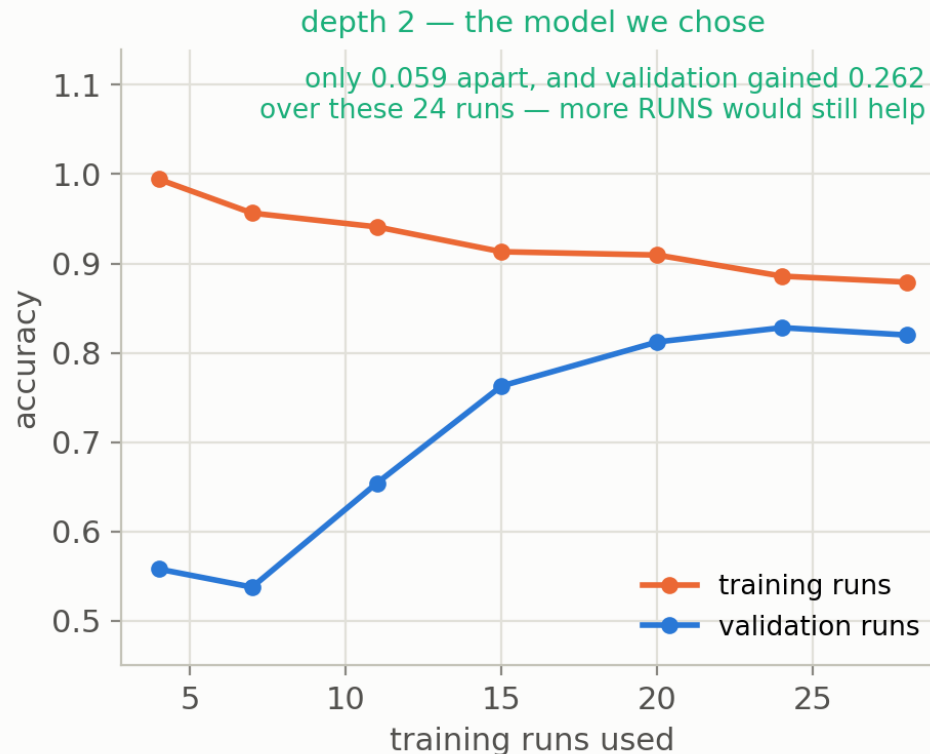


Both models ask to be made simpler. With only 28 independent training runs, complexity has nothing real left to fit.

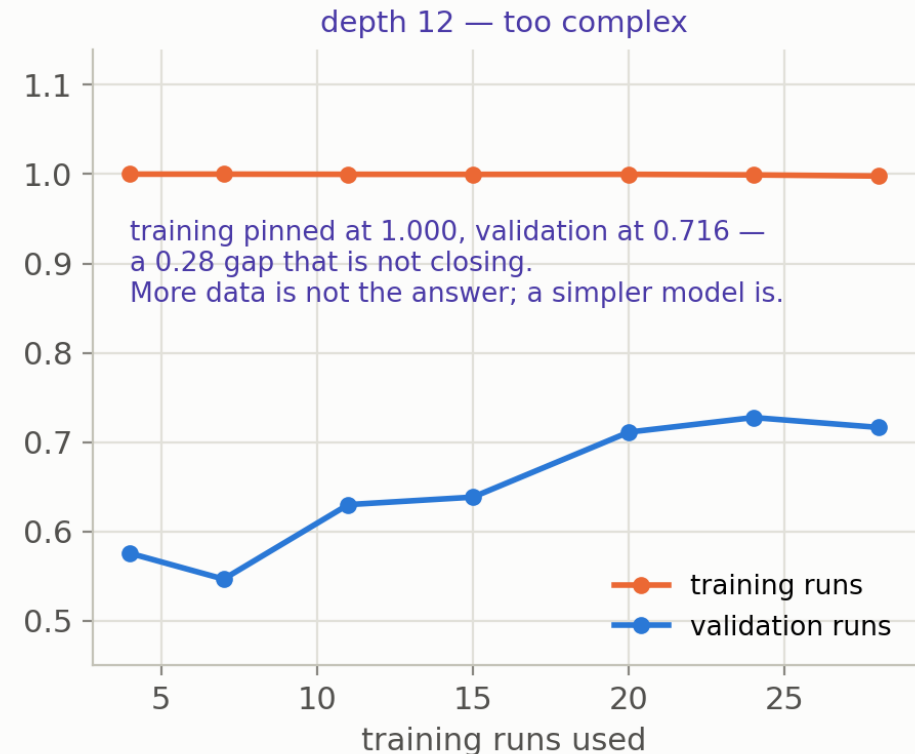
Both models ask to be made much simpler than you would guess: depth 2, and k = 401. With 28 independent training runs, extra capacity buys memorisation, not skill.

Would more data help?

- A validation curve varies the model at fixed data. A learning curve varies the data at a fixed model — and answers the question your manager will actually ask.



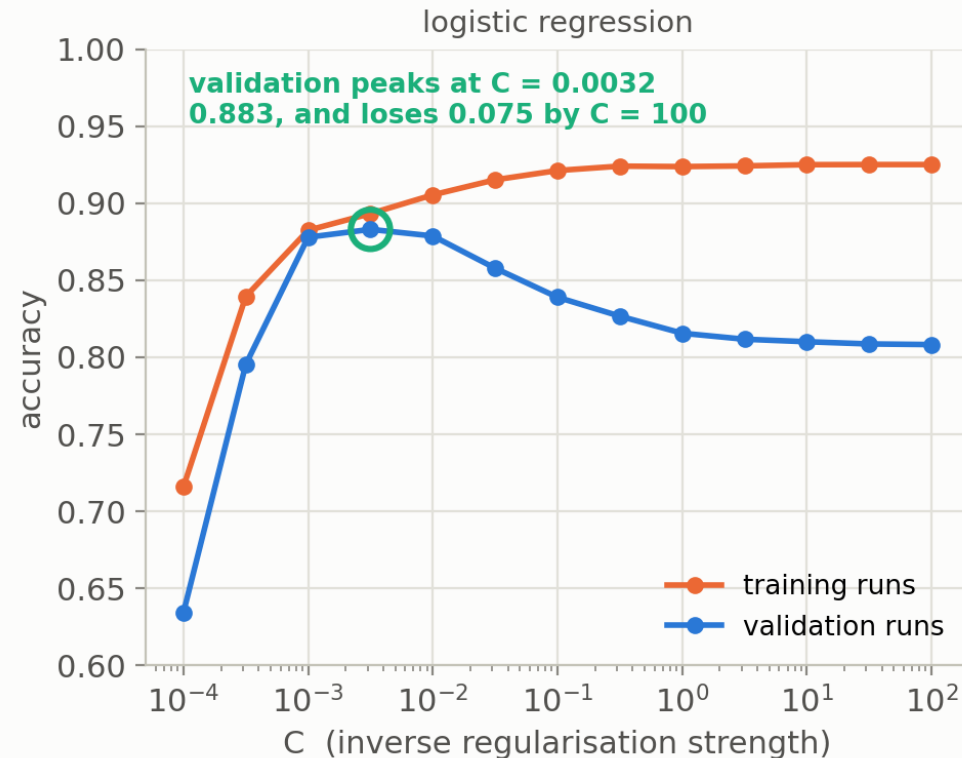
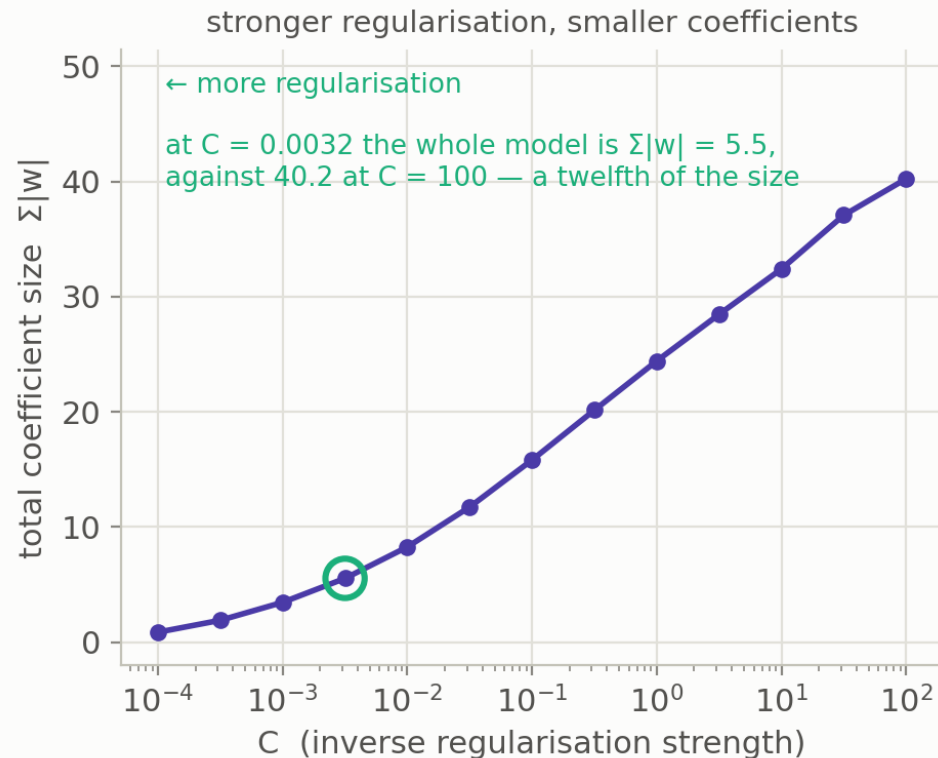
The x-axis counts RUNS, not rows. Adding more windows from a run you already have buys almost nothing.



Depth 2: curves 0.059 apart and validation still climbing — collect more runs. Depth 12: a 0.28 gap that is not closing — no amount of data will fix that.

Regularisation, and why C is backwards

- A penalty on the size of the weights makes the fit earn every unit of weight it uses. In scikit-learn the knob is C, the inverse strength — small C means more regularisation.



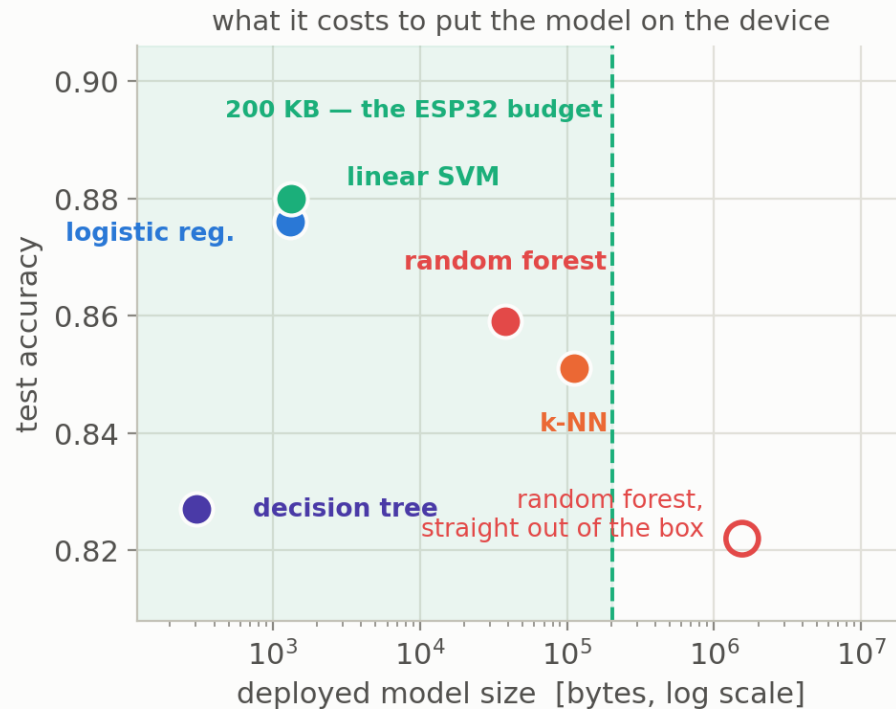
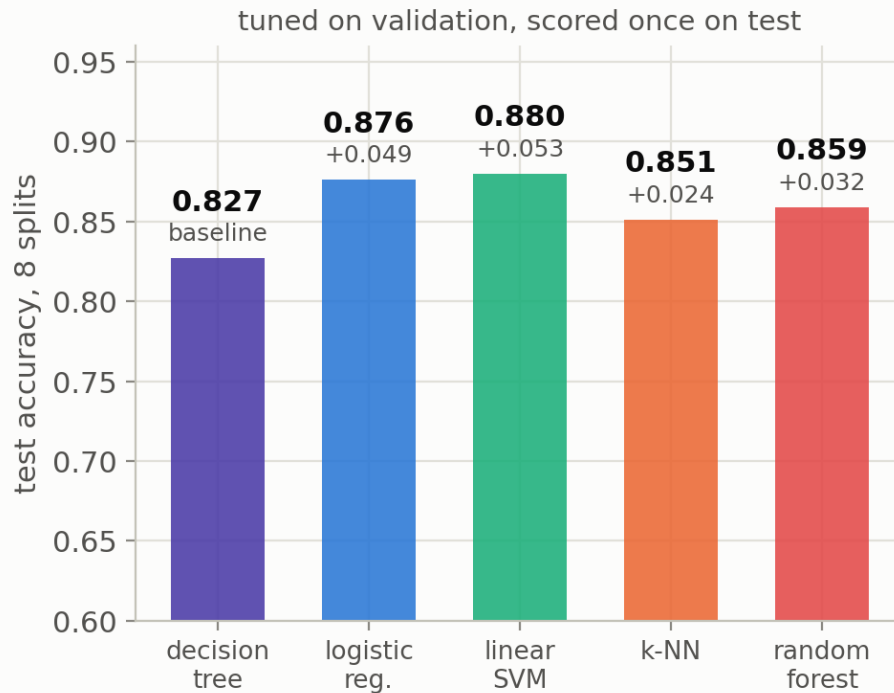
Training accuracy is flat across four decades of C. Validation is not — and only validation is telling you about machines you have not recorded.

Training accuracy is flat across four decades of C. Validation peaks at $C = 0.0032$ (0.883) and loses 0.075 by $C = 100$ — and the good model is also a twelfth the size.

RESULTS

The comparison — and the surprise

- Each family tuned on validation, refitted, scored once on test, over eight splits. Then measured for size, by actually exporting it with the tool Lecture 11 will use.



*Small print on the bars: the gain over the tree on the same splits, so run-to-run variance cancels.
The forest you get out of the box is 1,177× larger than the linear SVM and 0.058 less accurate.*

The linear SVM wins on accuracy (0.880) and on size (1322 B). The default random forest is 1,177× larger and 0.058 less accurate.

Accuracy is half the answer

- Size measured by exporting each model with m2cgen — the same tool Lecture 11 uses to turn a scikit-learn model into MicroPython.

model	knob	bytes	ops	accuracy
decision tree	d=2	303	2	0.827
logistic reg.	C=0.0032	1313	36	0.876
linear SVM	C=0.0001	1322	36	0.880
k-NN	k=401	111264	27816	0.851
random forest	d=2	38233	200	0.859
...untuned	—	1556017	1485	0.822

k-NN's problem is not size

At 109 KB it would fit in RAM. But it does 27,816 operations per decision against the linear SVM's 36 — and that cost grows every time you record another run.

The whole trained model

36 coefficients, 3 intercepts, 24 scaler constants. 63 numbers, 252 bytes of data, and a double loop to evaluate them.

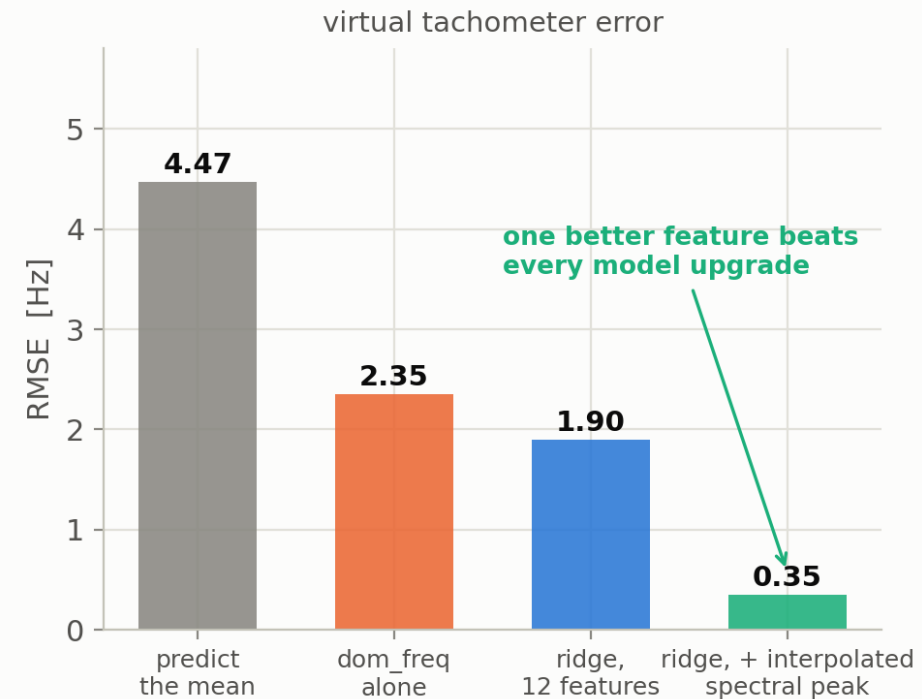
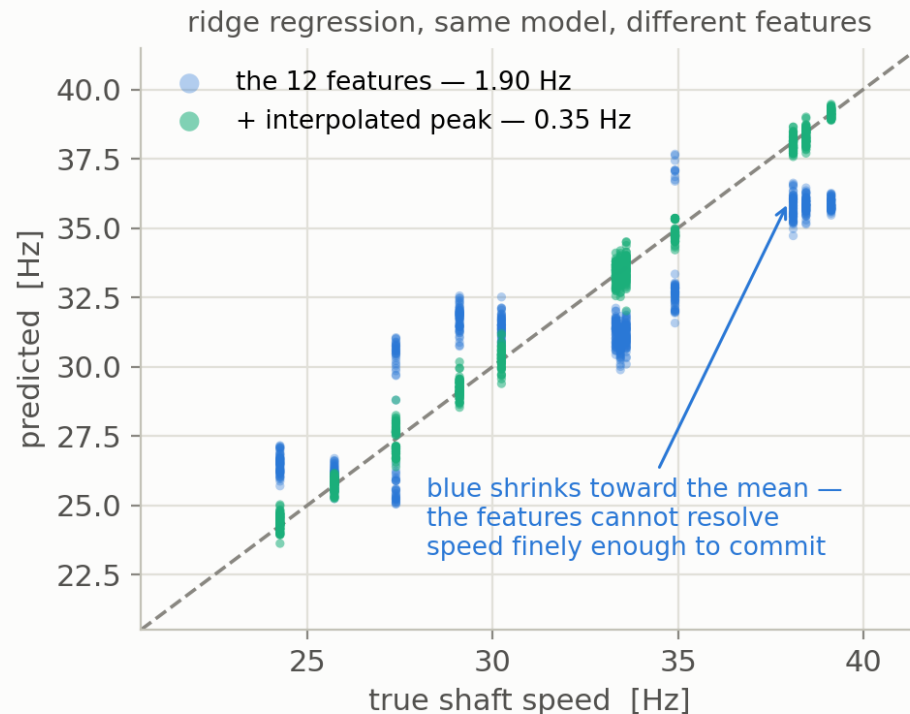
Why Lecture 11 will be short

Nothing here needs TensorFlow Lite, quantisation-aware training, or a C toolchain. It needs a for-loop in MicroPython.

A model whose inference cost grows with your dataset cannot be deployed to a device that must run unattended for years.

A tachometer the rig does not have

- Shaft speed sets the 1× line, the bearing defect frequencies and the expected vibration level. Computing it from the accelerometer alone is a soft sensor.



A random forest on the same 12 features reaches 2.15 Hz — no better. The 7.81 Hz FFT bin threw the resolution away before any model saw the data.

Ridge reaches 1.90 Hz; a random forest reaches 2.15 Hz. When every model plateaus together, stop tuning models — one interpolated spectral peak gets to 0.35 Hz.

Lab 9 — choose a model and defend the choice

Part A — reproduce

Run the notebook. Confirm the depth sweep, the learning curves and the regularisation path on your own machine.

Part B — your own comparison

Tune `min_samples_leaf` instead of `max_depth`. Add one model family of your choice. Report everything on shared splits with paired differences.

Part C — commit

Pick one model. Export `lecture9_model.json`. Write the paragraph that justifies it on accuracy, size and inference cost together.

```
$ python lab9.py
```

```
chosen C = 0.0032  validation 0.857  
TEST accuracy (opened once): 0.857
```

```
saved lecture9_model.json  
  36 coefficients  
   3 intercepts  
  24 scaler constants  
— 63 numbers total
```

Pace for the lab half

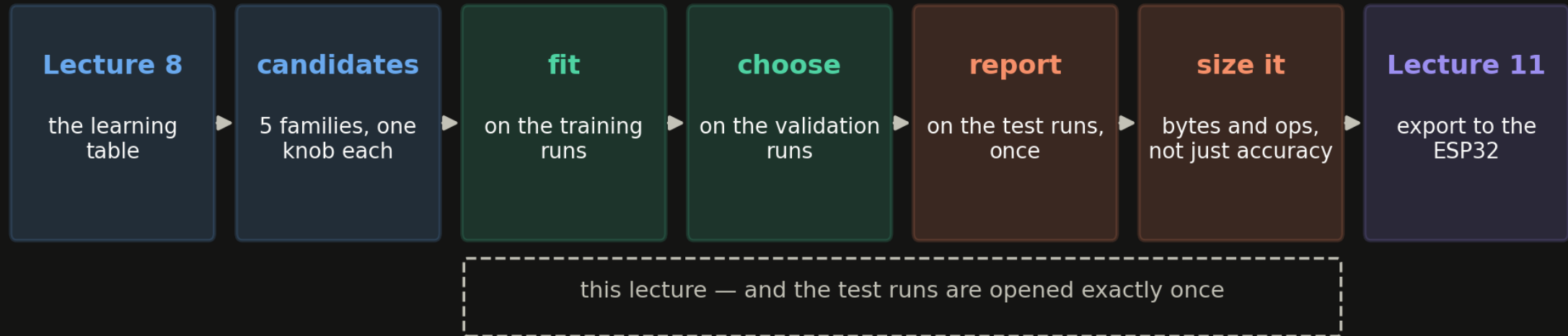
Notebook walk-through 60 min · your own comparison 90 min · write the justification 30 min.

Deliverables

Your comparison table with paired differences ·
`lecture9_model.json` · one paragraph defending the choice on all three axes · the test set opened exactly once, and you must say so.

Keep `lecture9_model.json`. Lecture 11 turns those 63 numbers into MicroPython and puts them back on the device that produced the features.

Capacity has to match the number of independent observations



1. Regression or classification — only the target column changes. Write down what a useless model scores first.
2. Three piles: fit on train, choose on validation, report on test once. 48 runs, 28 / 10 / 10.
3. Underfit: both curves low together. Overfit: a gap that does not close. Here depth 2 and $k = 401$ won.
4. Small C means more regularisation — and training accuracy will not tell you where to set it.
5. Compare on shared splits and difference the results: the paired standard error was five times smaller.
6. Measure bytes and ops beside accuracy. 1322 B beat 1520 KB on both.

Generated by Claude and customized by

